```
  ...
}
```

用户栈要等 thread_join() 中内核回收了线程之后才释放，否则线程可能还在用这块栈。

8.4.3 clone 为什么“返回两次”

fork() 会返回两次：父进程中返回子进程的 pid，子进程中返回 0，因为子进程的 trapframe 是父进程的完整拷贝，只把 x0 改成了 0。Linux 的 clone 系统调用语义相同，所以 glibc 的 clone() 包装函数要在汇编里处理两次返回：调用前把函数指针和参数存进不会被系统调用破坏的寄存器；系统调用返回非 0 说明还在原线程，直接返回调用者；返回 0 说明已经身处新线程，就取出函数指针调用它，函数结束后再调用 exit——新线程永远不会回到包装函数的调用者那里。

xv6-aarch64 的 clone(entry, arg, stack_top) 换了一种更简单的语义：内核直接把新线程的入口、参数和栈写进它的 trapframe，新线程第一次回到用户态时就从 entry 开始执行，用户态不需要任何汇编包装：

```
int threadclone(uint64 entry, uint64 arg, uint64 stack_top)
{
  ...
  if(entry == 0 || stack_top < 16 || (stack_top & 15) != 0)
    return -1;                          // 栈必须 16 字节对齐
  ...
  vm->refcount++;                       // 共享同一个地址空间
  vmspace_put(np->vm);
  np->vm = vm;
  np->is_thread = 1;
  np->tgid = p->tgid;

  *(np->trapframe) = *(p->trapframe);
  np->trapframe->elr = entry;           // eret 之后从这里开始
  np->trapframe->sp  = stack_top;       // 新的用户栈
  np->trapframe->x0  = arg;             // 第一个参数
  np->trapframe->x30 = 0;               // 不允许“返回”
  ...
  make_runnable(np, 0);
  return tid;
}
```

x30 被清零，线程函数如果直接 ret 就会跳到地址 0 出错，所以用户库的 thread_boot() 包了一层，保证函数返回后调用 texit()。线程退出后变成 ZOMBIE，等 tjoin() 回收——与进程的 exit/wait 是同一种约定：先保留“尸体”，让等待者能取到退出状态，再由等待者释放资源。

8.4.4 多核下的细节

- sbrk() 修改共享的页表时持有 vmspace.lock，改完执行 tlbi vmalle1is 广播失效，覆盖其他核上正在运行的同组线程；
- 用户态的 K&R malloc 加了一把基于 ldaxr/stxr 的锁；
- exec() 只在线程组只剩调用者时才允许执行；exit() 先终止并回收同组线程；kill(pid) 以 TGID 为目标，标记并唤醒整组；